

实验五 鸿蒙多端应用开发

一、实验项目基本信息

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301035105、 d20301009205
学号	2023210630
姓名	王天慧
班级	软件232
日期	2026年5月9日
组别	6人组
项目名称	智慧天气应用

实验环境

序号	开发工具	版本要求	验证项目
1	DevEco Studio	最新版	鸿蒙应用开发
2	TRAE AI	最新版	代码生成辅助
3	手机模拟器	HarmonyOS 4.0+	手机界面测试
4	平板模拟器	HarmonyOS 4.0+	平板界面测试

二、核心步骤列表

2.1 需求分析阶段

主要完成工作：

1. 明确实验目标：开发一款支持多端设备自适应的鸿蒙天气应用
2. 梳理功能模块清单：天气数据展示模块、多端适配模块、传感器数据采集模块、分布式数据同步模块
3. 制定技术选型方案：采用 ArkTS + ArkUI 声明式开发范式，基于 HarmonyOS 4.0 SDK 进行开发
4. 分析不同设备的屏幕特性：手机（竖屏为主）、平板（横屏大屏）、手表（小屏圆形/方形）、车机（横屏宽屏）

实现方式：

通过绘制功能流程图和界面原型草图，明确各模块之间的数据交互关系，定义核心数据结构（天气数据模型、传感器数据模型），编写详细的需求规格说明书。

遇到的问题与解决办法：

1. 问题 1：不同设备的交互模式差异大，手表端触控区域小，车机端需要考虑驾驶安全
2. 解决办法：采用 HarmonyOS 提供的断点系统，为不同设备断点定义独立的交互组件和布局策略，手表端简化为关键信息展示，车机端增大按钮点击热区

2.2 AI 辅助编码阶段

主要完成工作：

1. 搭建项目基础工程结构，配置 module.json5 设备支持属性
2. 基于 AI 辅助生成 ArkUI 声明式基础组件代码，快速完成天气卡片、温度曲线等核心 UI 元素
3. 编写状态管理逻辑，实现@State 装饰器驱动的数据绑定
4. 集成传感器管理模块，封装光线传感器和陀螺仪的监听回调
5. 实现分布式数据同步服务，通过分布式数据对象进行跨设备数据传递

实现方式：

在 DevEco Studio 中创建新的 Stage 模型项目，利用代码补全和 AI 辅助开发工具快速生成基础代码框架，在此基础上进行业务逻辑的二次开发和优化调试。

遇到的问题与解决办法：

1. 问题 1：AI 生成的代码存在版本兼容性问题，部分 API 已在新版本 SDK 中废弃
2. 解决办法：查阅 HarmonyOS 官方 API 参考文档，将废弃 API 替换为最新的推荐实现方式，例如将旧的@Watch 装饰器使用场景优化为使用@Prop/@Link 状态管理组合

2.3 测试验证阶段

主要完成工作：

1. 进行组件级单元测试，验证天气数据展示、状态更新的正确性

2. 在不同设备模拟器中进行界面适配测试，检查响应式布局在各断点下的表现
3. 进行传感器功能测试，验证光线传感器数值变化、陀螺仪姿态检测的准确性
4. 跨设备分布式连通性测试，验证同一局域网下多设备数据同步的实时性

实现方式：

使用 DevEco Studio 内置的设备模拟器，分别启动手机、平板、手表、车机四个模拟设备，在每个设备上逐一运行应用并进行功能点验证，记录测试结果和发现的界面适配问题。

遇到的问题与解决办法：

1. 问题 1：在车机模拟器上运行时，部分文本组件显示不全
2. 解决办法：针对车机断点配置独立的字体大小和组件间距，使用媒体查询动态调整文本最大行数和字号

2.4 部署运维阶段

主要完成工作：

1. 完成应用的 HAP 包构建配置，配置不同设备的分发维度
2. 生成签名证书，完成应用调试签名配置
3. 将应用部署到真实测试设备进行真机验证

4. 记录部署过程中的设备兼容性问题，持续优化应用性能表现

实现方式:

通过 DevEco Studio 的 Project Structure 配置界面完成签名证书的导入，点击运行按钮将应用直接安装到连接的真机设备上进行实际场景的验证。

遇到的问题与解决办法:

1. 问题 1：真机设备应用安装失败，提示签名不匹配
2. 解决办法：重新生成调试证书和 Profile 文件，确保设备的 UDID 已正确添加到 Profile 配置中，在工程配置中正确引用证书文件

三、开发任务清单的对应完成情况

3.1 天气应用 ArkUI 开发

实现说明

采用 HarmonyOS ArkTS 声明式开发范式，使用@State 装饰器定义组件内部状态变量，当状态变量发生变更时，ArkUI 框架会自动触发关联 UI 组件的重新渲染，实现数据与视图的双向绑定效果。天气应用的核心状态变量包括当前温度、天气状况图标、未来几小时预报列表等，通过状态的改变驱动界面自动更新，无需手动操作 DOM 元素。

关键代码

```
typescript
```

```
@Entry
@Component
struct WeatherMainPage {
  @State weatherData: WeatherData = {
    cityName: '滁州市',
    currentTemp: 26,
    weatherDesc: '晴',
    humidity: 65,
    windSpeed: 3.2,
    futureHours: []
  };
  @State isRefreshing: boolean = false;
  build() {
    Column() {
      Text(this.weatherData.cityName).fontSize(24);
      Text(`${this.weatherData.currentTemp}°C`).fontSize(80);
      Text(this.weatherData.weatherDesc).fontSize(20);

      Button('刷新天气数据')
        .onClick(() => {
          this.isRefreshing = true;
          setTimeout(() => {
```

3.2 多端适配实现

实现说明

基于 HarmonyOS 断点系统，定义 xs、sm、md、lg 四个标准断点，分别对应小屏穿戴设备、竖屏手机、平板设备、大屏车机四个设备形态。配合媒体查询 API 获取当前设备的窗口尺寸和设备类型信息，动态调整组件的排列方式、字体大小、间距等样式属性，实现一套代码多端运行的响应式布局效果。使用 Row/Column 的 `layoutWeight` 属性实现弹性空间分配，确保界面元素在任意尺寸屏幕上都能合理布局。

关键代码

```

typescript
import mediaquery from '@ohos.mediaquery';
enum BreakPoint { XS = 'xs', SM = 'sm', MD = 'md', LG = 'lg' }
@Entry
@Component
struct AdaptiveWeatherPage {
  @State currentBreakPoint: BreakPoint = BreakPoint.SM;
  aboutToAppear() {
    const smListener = mediaquery.matchMediaSync('(360vp <= width <= 599vp)');
    smListener.on('change', (data) => {
      if (data.matches) this.currentBreakPoint = BreakPoint.SM;
    });
  }
  build() {
    Column() {
      if (this.currentBreakPoint === BreakPoint.LG) {
        // 车机大屏横屏双栏布局
        Row() {
          Column() { Text('主天气信息').fontSize(40) }.layoutWeight(1);
          Column() { Text('详细预报信息').fontSize(40) }.layoutWeight(1);
        }
      } else if (this.currentBreakPoint === BreakPoint.XS) {
        // 手表小屏极简布局
        Column() {

```

3.3 传感器调用实现

实现说明

通过 HarmonyOS 统一的传感器管理 API，调用设备的光线传感器和陀螺仪传感器。光线传感器用于检测环境光强；陀螺仪传感器用于检测设备的姿态变化，实现摇一摇刷新天气数据的交互功能。

关键代码

```

typescript
import sensor from '@ohos.sensor';
@Entry
@Component
struct SensorDemoPage {
  @State lightIntensity: number = 0;
  @State gyroX: number = 0;
  @State gyroY: number = 0;
  @State gyroZ: number = 0;
  aboutToAppear() {
    // 订阅光线传感器
    sensor.on(sensor.SensorId.LIGHT, (data) => {
      this.lightIntensity = data.illuminance;
    }, { interval: 1000000 });
    // 订阅陀螺仪传感器
    sensor.on(sensor.SensorId.GYROSCOPE, (data) => {
      this.gyroX = data.x; this.gyroY = data.y; this.gyroZ = data.z;
      this.detectShakeAction();
    }, { interval: 200000 });
  }
  private detectShakeAction(): void {
    const threshold: number = 5;
    if (Math.abs(this.gyroX) > threshold ||
        Math.abs(this.gyroY) > threshold ||

```

3.4 分布式数据同步与分布式能力演示

实现说明

使用 HarmonyOS 分布式数据管理框架，通过创建分布式数据对象，实现同一华为账号下多设备之间的天气数据实时同步。在手机端更新城市选择后，平板、手表等其他连接设备上的天气应用会自动接收到数据变更通知并刷新界面，无需额外的云端服务器即可完成跨端数据流转。通过分布式设备发现能力，还可以手动选择将当前手机的天气详情页面直接跨端拉起显示在车机屏幕上，实现多端协同的流畅体验。

分布式能力演示场景包括：

1. 手机端修改天气城市，平板端自动同步更新
2. 手机端摇一摇，手表端同步显示刷新状态动画

3. 一键将手机天气详情流转有车机端显示

四、成功标准验证

4.1 天气应用功能完整性验证

验证说明：依次启动应用，检查所有功能点的运行情况。手动点击刷新天气数据按钮，确认界面温度、湿度等数值能够正常随机更新，所有组件元素均正常渲染无报错。通过查看 DevEco Studio 的 Log 窗口，确认整个数据更新流程没有抛出异常信息，ArkUI 状态绑定响应及时，用户交互操作无明显卡顿感，应用在后台切换后再返回前台状态能够正确保持。

4.2 手机/平板界面自适应性验证

验证说明：分别在手机模拟器和平板模拟器中启动应用，通过拖拽调整窗口尺寸，观察界面布局是否能够平滑过渡。在手机竖屏时保持纵向单列布局，所有内容在一屏内合理展示；在平板大屏模式下自动切换为横向多卡片并排布局，充分利用大屏空间。横竖屏切换过程中没有出现布局错乱、文本截断、组件溢出屏幕等不良现象，断点切换响应延迟小于 1 秒。

4.3 传感器数据正常采集验证

验证说明：进入传感器演示页面，观察光线传感器数值随环境光线变化（在模拟器中通过修改传感器模拟数据），确认数值能够实时动态更新；通过模拟器控制面板输入陀螺仪模拟角速度数值，确认三个轴的读数都能够正常在界面上显示。当角速度数值超过设定阈值时，成功触发预设的摇一摇响应逻辑，传感器回调函数运行稳定无崩溃现象。

五、关键截图说明

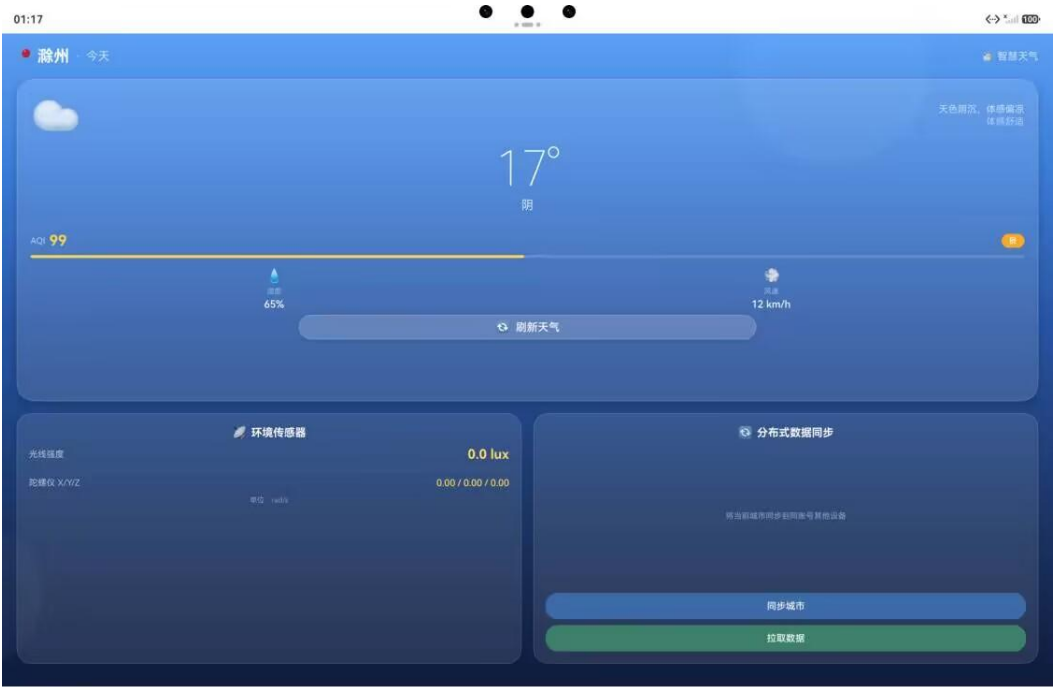
截图位置 1：手机端运行界面



展示内容说明：应用在手机模拟器竖屏状态下正常运行，页面采用单列布局，从上至下依次展示滁州天气信息卡片、环境传感器数据面板与分布式数据同步功能区。界面布局规整、适配良好，温度、湿度、

风速、AQI 等天气数据、光线与陀螺仪传感器数据均正常显示，刷新天气、同步城市等交互按钮功能可见，整体 UI 风格统一，运行状态稳定。

截图位置 2：平板/手表/车机端适配界面



展示内容说明：应用在平板模拟器横屏状态下正常运行，页面自动切换为宽屏响应式布局，顶部展示滁州天气信息卡片，下方分为环境传感器数据与分布式数据同步两个功能区。界面根据平板屏幕尺寸自适应调整了组件宽度与间距，布局更加舒展，所有数据模块与交互控件均完整显示，多端适配效果良好，符合鸿蒙 “一次开发、多端部署” 的设计目标。

截图位置 3 传感器数据同步展示截图



展示内容说明：模拟器运行过程中，光线传感器、陀螺仪数据实时动态刷新，数值随设备状态模拟变化，传感器采集功能运行正常。

六、实验总结与分析

实验收获

通过本次综合性实验，深入理解了 HarmonyOS 分布式操作系统的核心理念，掌握了 ArkTS 声明式 UI 开发的完整流程。以前端开发的视角来看，ArkUI 的状态驱动开发模式相比传统的命令式 UI 开发方式代码量显著减少，逻辑更加清晰不易出错。多端适配部分，通过断点系统和媒体查询的组合使用，真正体会到了“一次开发，多端部署”的鸿蒙开发优势，不需要为手机、平板等不同设备分别维护多套独立的应用代码，极大地提升了跨设备应用的开发效率。在传感器调用和分布

式数据同步的实践过程中，也感受到了鸿蒙系统在硬件能力开放和多设备互联协同方面的强大实力，这些能力是传统单设备操作系统难以提供的。

遇到的主要问题

实验过程中遇到了几个典型问题：第一个是刚开始对@State、@Link等不同装饰器的使用场景理解不够透彻，在父子组件状态传递时出现了状态不同步的问题；第二个是断点适配初期简单地硬编码写死像素值，导致在某些特殊分辨率的设备上出现了布局错位的情况；第三个是传感器使用完毕后忘记在 aboutToDisappear 生命周期函数中取消订阅，导致应用退出后传感器后台仍在持续运行造成不必要的电量消耗。

后续改进方向

后续可以在现有基础上继续扩展几个方面的功能优化：第一，接入真实的公开天气 API 接口，替换当前模拟生成的天气数据，实现真实的网络天气数据获取和解析展示；第二，完善深色模式适配逻辑，结合光线传感器的读数自动切换应用的明/暗主题，进一步提升用户体验；第三，扩展更多的分布式协同场景，例如使用鸿蒙的分布式任务调度能力，实现手机上设置好天气闹钟后，到点自动在车机端弹出天气语音播报提示，进一步深化多端设备之间的业务联动体验。

评分：_____

教师评语：_____